

**Assignment 7**

# **Secure Network Application Development Using TCP**

Multi-Client Secure Chat Application — Report

Submitted by:

Name: Abhishek Das

Roll No.: EEB24021

Date: July 16, 2026

## 1. Objective

This assignment extends the GUI-based multi-client TCP chat application built in Assignment 6 by adding practical application-level security mechanisms. The focus is on authentication, secure password storage, session control and input validation — not on advanced cryptography. The enhanced application allows multiple clients to connect concurrently over a Mininet-emulated network, authenticate with a username and password, exchange broadcast and private messages, and terminate sessions securely.

## 2. Security Features Implemented

### 2.1 User Authentication (Task 1)

Every client must authenticate before joining the chat. On connection, the client sends a single handshake payload of the form `username\xlfpassword` (using an ASCII unit-separator delimiter so usernames/passwords cannot break the parser). The server's `authenticate()` function validates the credentials before granting access; unauthenticated sockets are never added to the active client list.

### 2.2 Secure Password Storage (Task 2)

Passwords are never stored or logged in plaintext. The server hashes every password with SHA-256 (`hashlib.sha256`) before writing it to `users.json`, and compares hashes — never raw passwords — during login:

```
def hash_password(password):
    return hashlib.sha256(password.encode("utf-8")).hexdigest()
```



```
vboxuser@ubuntu-26:~/as7$ cat users.json
{
  "abhishek": "03ac674216f3e15c761ee1a5e255f067953623c8b388b4459e13f978d7c846f4",
  "rahul": "5994471abb01112afcc18159f6cc74b4f511b99806da59b3caf5a9c173cacfc5"
}vboxuser@ubuntu-26:~/as7$
```

*Figure 1: users.json storing only SHA-256 password hashes, never plaintext.*

## 2.3 Duplicate Login Prevention (Task 3)

The server maintains an in-memory set, `active_usernames`, of usernames that currently hold an open session. A new login attempt for a username already present in this set is rejected with `AUTH_FAIL`: This user is already logged in elsewhere, and the event is written to the security log as `AUTH_DUPLICATE_LOGIN_BLOCKED`.

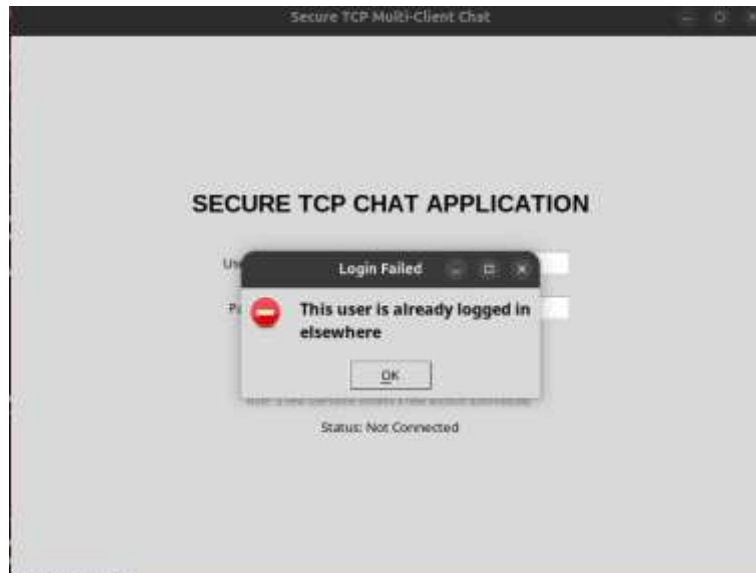


Figure 2: Client rejected because the username is already logged in from another session.

## 2.4 Input Validation (Task 4)

All input is validated before it is trusted:

- Usernames must match `^[A-Za-z0-9_]{3,20}$` (3-20 alphanumeric/underscore characters).
- Passwords must be at least 4 characters long.
- Chat messages longer than 500 characters (`MAX_MESSAGE_LENGTH`) are rejected on both the client and server.
- Any command beginning with "/" that is not `/list`, `/logout`, or `/msg` is rejected as an unsupported command.
- Malformed authentication payloads (missing the delimiter) are rejected immediately.

## 2.5 Failed Login Protection (Task 5)

The server tracks consecutive failed login attempts per username. After 5 consecutive failures (MAX\_FAILED\_ATTEMPTS), the account is locked for 60 seconds (LOCKOUT\_SECONDS); further attempts during that window receive AUTH\_LOCKED:<seconds\_remaining> instead of being re-checked against the password, preventing brute-force guessing.

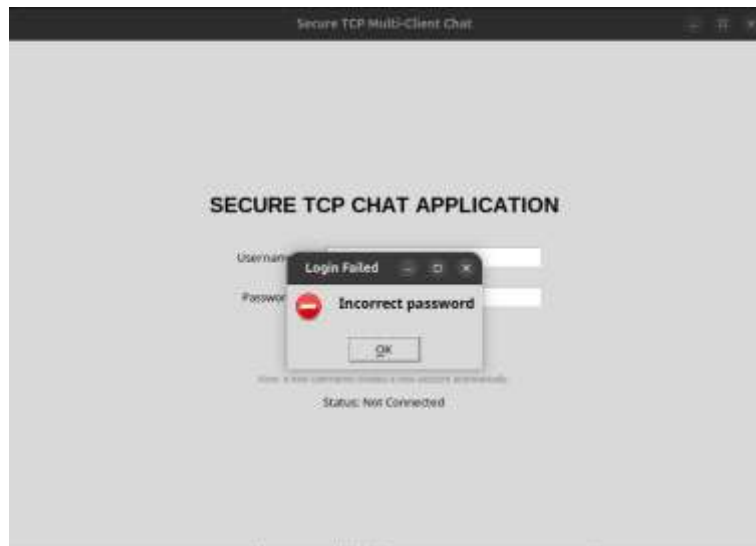


Figure 3: Login rejected with "Incorrect password" before the lockout threshold is reached.



Figure 4: Account temporarily locked after 5 consecutive failed attempts.

## 2.6 Session Management & Secure Logging (Task 6)

Clients can end a session explicitly with `/logout`, or the server will auto-terminate a session after 5 minutes (`SESSION_TIMEOUT`) of inactivity, detected via a rolling `recv()` timeout. Every security-relevant event — account creation, login success/failure, lockout, duplicate-login blocks, logout, disconnects and session timeouts — is appended to `security_log.txt` with a timestamp, event type, username and IP address. Passwords are never written to this log.

```

]vboxuser@ubuntu-26:~/as7$ cat security_log.txt
2026-07-16 14:29:27 | ACCOUNT_CREATED | user=abhishek | ip=10.0.0.2 |
2026-07-16 14:32:40 | ACCOUNT_CREATED | user=rahul | ip=10.0.0.3 |
2026-07-16 14:34:17 | AUTH_DUPLICATE_LOGIN_BLOCKED | user=abhishek | ip=10.0.0.4 |
2026-07-16 14:34:50 | AUTH_DUPLICATE_LOGIN_BLOCKED | user=abhishek | ip=10.0.0.4 |
2026-07-16 14:35:11 | AUTH_DUPLICATE_LOGIN_BLOCKED | user=rahul | ip=10.0.0.4 |
2026-07-16 14:35:18 | LOGOUT | user=rahul | ip=10.0.0.3 |
2026-07-16 14:35:18 | DISCONNECTED | user=rahul | ip=10.0.0.3 |
2026-07-16 14:35:25 | LOGIN_FAILED | user=rahul | ip=10.0.0.3 |
2026-07-16 14:35:40 | LOGIN_FAILED | user=rahul | ip=10.0.0.3 |
2026-07-16 14:35:42 | LOGIN_FAILED | user=rahul | ip=10.0.0.3 |
2026-07-16 14:35:43 | LOGIN_FAILED | user=rahul | ip=10.0.0.3 |
2026-07-16 14:35:45 | ACCOUNT_LOCKED | user=rahul | ip=10.0.0.3 | too many failed attempts, locked for 60s
]vboxuser@ubuntu-26:~/as7$

```

Figure 5: `security_log.txt` showing authentication and session events with no passwords recorded.

## 3. System Architecture

The application follows a client-server architecture over TCP, keeping networking logic independent of the GUI as required:

- `server.py` — multi-threaded TCP server (one thread per client) that owns authentication, session state, input validation, message routing (broadcast/private) and security logging.
- `client_gui.py` — Tkinter GUI that collects credentials, performs the login handshake, and renders chat/roster once authenticated. All socket I/O is isolated from widget code and run on a background thread so the UI stays responsive.
- `users.json` — persistent credential store (username → SHA-256 hash).
- `security_log.txt` / `chat_history.csv` — append-only logs for security auditing and chat history.

The network itself is emulated using Mininet with a single switch and 5 hosts (`sudo mn --topo single,5`), verified with `nodes`, `net` and `pingall` before running the application:

```

*** Configuring hosts
h1 h2 h3 h4 h5
*** Starting controller

*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> nodes
available nodes are:
h1 h2 h3 h4 h5 s1
mininet>

```

Figure 6: Mininet nodes — hosts `h1-h5` and switch `s1`.

```
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
h3 h3-eth0:s1-eth3
h4 h4-eth0:s1-eth4
h5 h5-eth0:s1-eth5
s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:h3-eth0 s1-eth4:h4-eth0 s1-eth5:h5-eth0
mininet>
```

Figure 7: Mininet net — host-to-switch interface mapping.

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5
h2 -> h1 h3 h4 h5
h3 -> h1 h2 h4 h5
h4 -> h1 h2 h3 h5
h5 -> h1 h2 h3 h4
*** Results: 0% dropped (20/20 received)
mininet>
```

Figure 8: pingall confirms 0% packet loss between all hosts (20/20 received).

## 4. Implementation

Key implementation details, in addition to the security mechanisms above:

### 4.1 Authentication Handshake

On connect, `authenticate(conn, addr)` reads the raw handshake, validates format/username/password, checks logout status, checks for a duplicate active session, then either registers a new account or verifies an existing one — replying with `AUTH_OK`, `AUTH_OK:REGISTERED`, `AUTH_FAIL:<reason>`, or `AUTH_LOCKED:<seconds>`.

### 4.2 Concurrency & Shared State

A single `threading.Lock (state_lock)` guards all shared dictionaries/sets (`clients`, `active_usernames`, `failed_attempts`) so concurrent client threads cannot corrupt session state.

### 4.3 GUI Flow

`client_gui.py` shows a login screen first; on `AUTH_OK` it switches to the chat screen (message pane, online-user list, send/private/refresh/logout controls) and starts a background thread to receive server pushes. `AUTH_FAIL` and `AUTH_LOCKED` responses are surfaced as message boxes and the socket is cleanly closed so the user can retry.

## 5. Testing

The application was tested end-to-end on the Mininet topology with multiple simulated hosts acting as clients connecting to the server on `h1`.

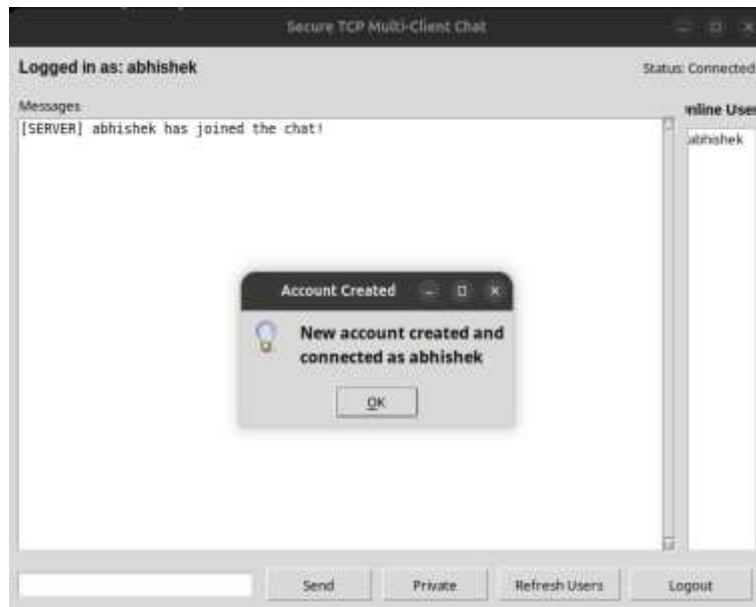


Figure 9: New account auto-created on first login for a new username.



Figure 10: Multiple users (abhishek, rahul) online simultaneously with join notifications.



Figure 11: Broadcast chat messages visible to all connected clients.

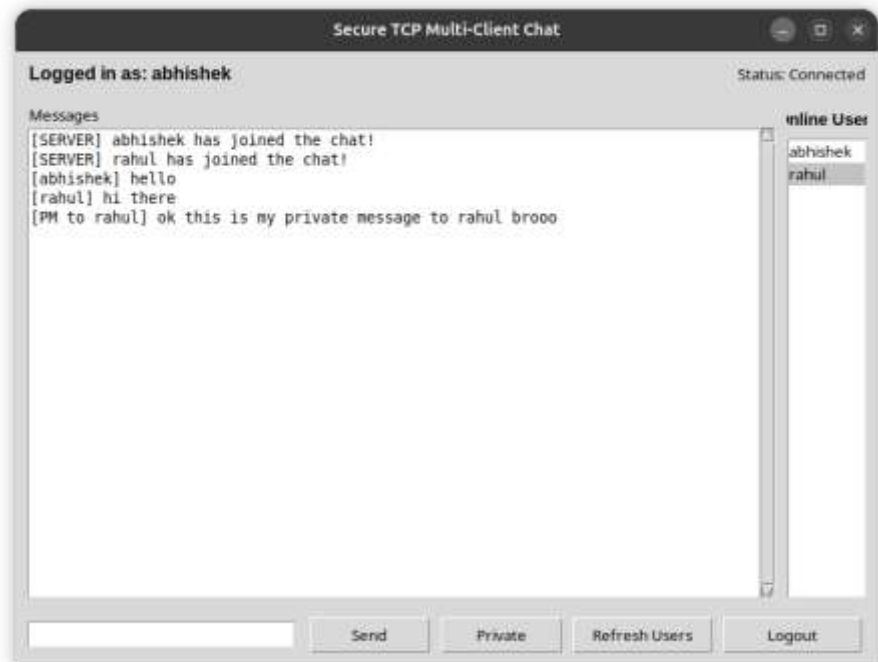


Figure 12: Private message (/msg) delivered only to the target user.

## 6. Wireshark Verification (Task 7)

Traffic on tcp.port == 5000 was captured on the switch interface (s1-eth1) to verify the TCP session lifecycle and confirm that authentication events are observable at the network layer.

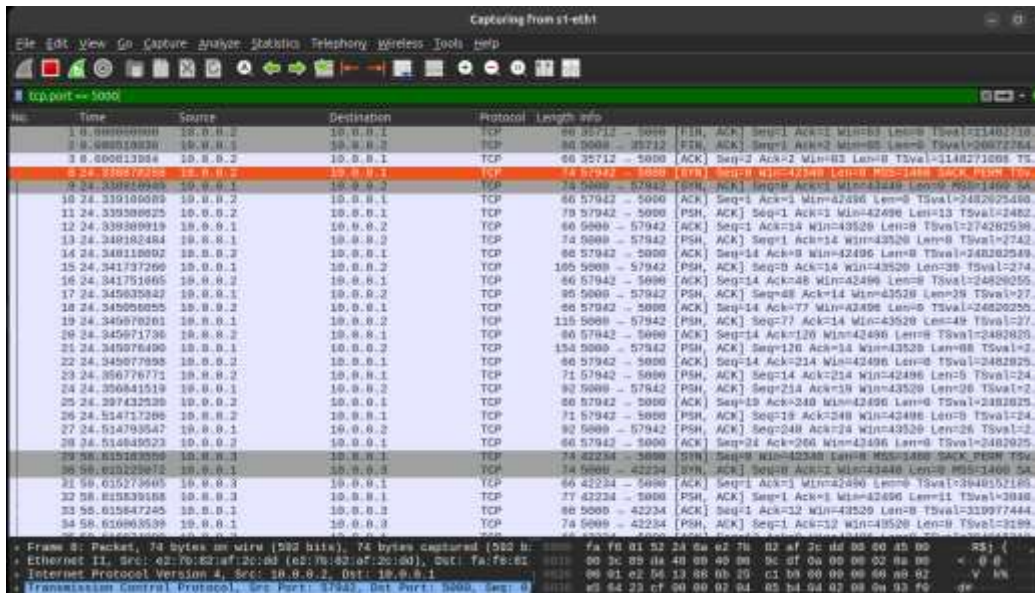


Figure 13: TCP three-way handshake (SYN, SYN-ACK, ACK) establishing a client session before authentication.

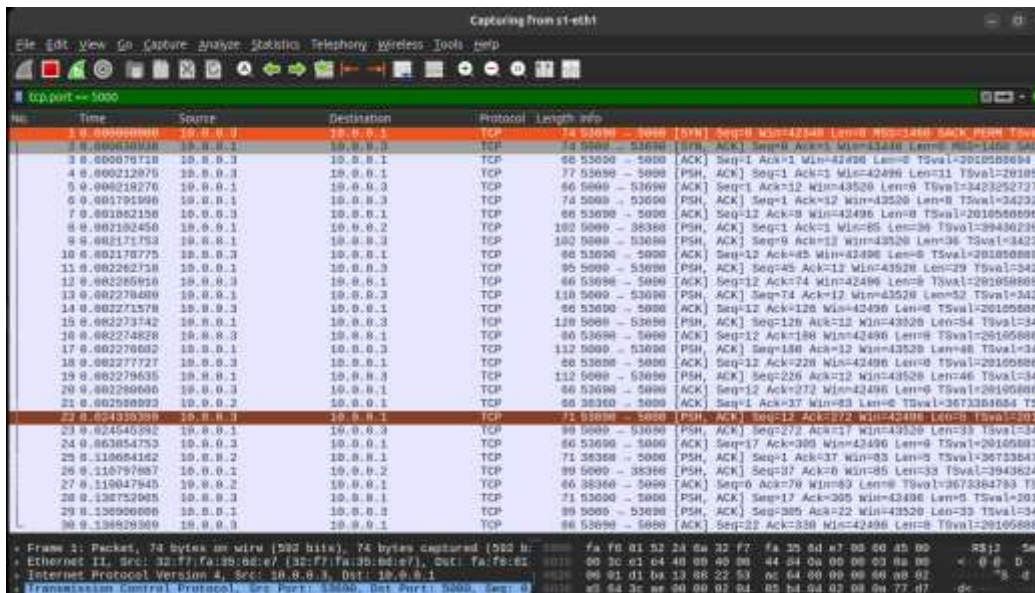


Figure 14: Full session capture for a successful login — handshake, PSH/ACK data exchange, and FIN teardown.

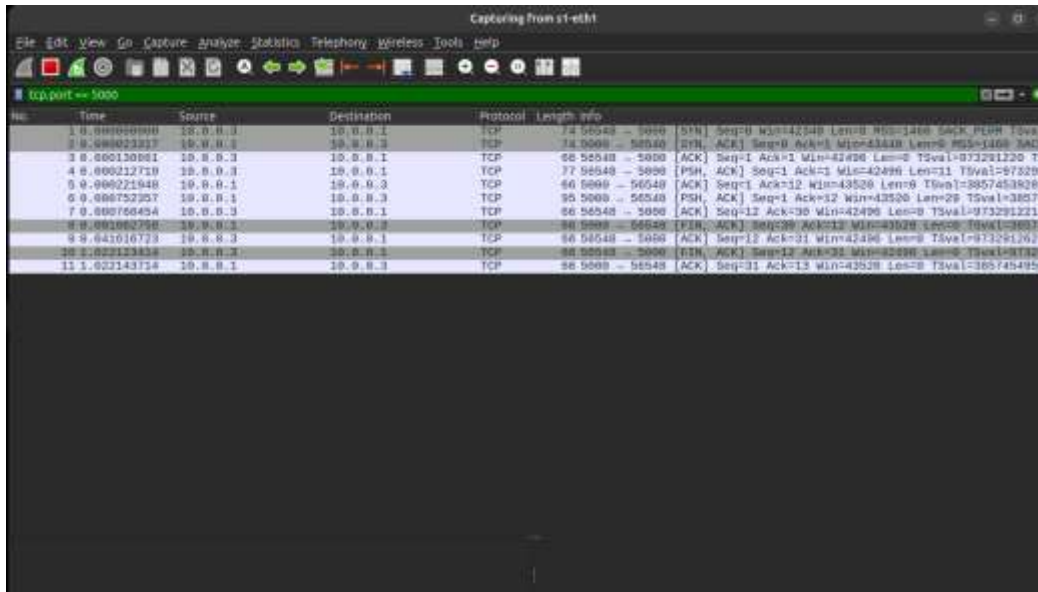


Figure 15: Capture showing a rejected login attempt (short PSH/ACK exchange followed by connection close).

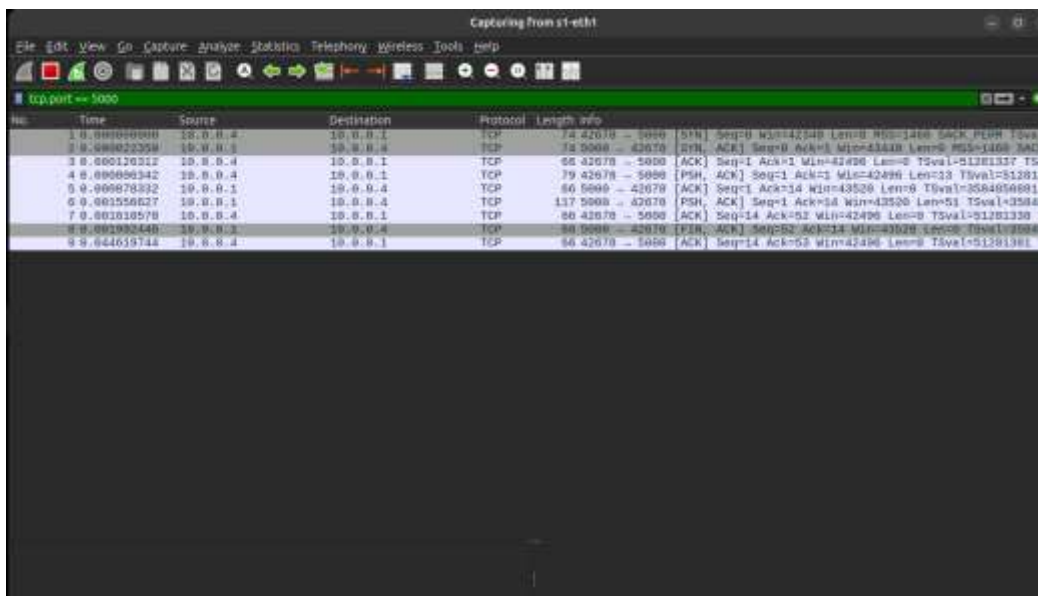


Figure 16: Capture showing a duplicate-login attempt being rejected while the original session remains active.

Note: passwords are visible as plaintext bytes in these captures because the assignment scope is application-layer security (hashing at rest, not transport encryption). A production deployment would additionally wrap the socket in TLS to protect credentials in transit — see the reflection notes for further discussion.

## 7. Conclusion

The enhanced chat application meets all functional and security requirements of Assignment 7: authenticated, hashed-credential logins; prevention of duplicate sessions and brute-force attempts; strict input validation; automatic session timeout with clean logout; and a tamper-evident, password-free security log. Wireshark captures confirm the expected TCP behaviour and authentication flow across the Mininet topology. The main follow-up improvement, noted in the handwritten reflection, would be adding transport-layer encryption (TLS) so credentials are not observable in plaintext on the wire.

## Appendix: Rubric Coverage

| Component                           | Marks |
|-------------------------------------|-------|
| User Authentication                 | 20    |
| Secure Password Storage             | 15    |
| Duplicate Login Prevention          | 10    |
| Input Validation                    | 15    |
| Failed Login Protection             | 10    |
| Session Management & Secure Logging | 10    |
| Wireshark Verification              | 10    |
| GitHub Update                       | 5     |
| Handwritten Reflection              | 5     |
| Total                               | 100   |